

FRIDAY — Build Journal & Obsidian Dev Loop

Combined technical specification · companion addendum to the Autonomous Build Playbook v1.0

Prepared for: Ethan Soliven · Date: July 12, 2026 · Vault: Scientia (C:\Users\ethan\Documents\Scientia)

1. How it fits — two agents, one vault, one UI

FRIDAY and Claude Code are the same idea at two layers over **one Obsidian vault**. **FRIDAY (runtime)** is the app you are building: it reads and writes the vault through the single-writer path `backend/writer.py` under suggest-then-approve, and reasons with its own in-process Claude Agent SDK MCP. **Claude Code (dev-time)** is the agent that builds FRIDAY and your other projects — this document adds a loop that records those build sessions and project context into the very vault FRIDAY indexes.

The payoff is compounding: FRIDAY's second brain ends up containing the record of how FRIDAY (and everything else) got built, and can answer questions about it. Crucially, this is not a headless backend feature — **FRIDAY is a web app** (Next.js 14 dashboard with Today, Tasks, Chat, Study, Review, and Insights pages), so the build journal gets a **first-class UI surface** (Section 5), not just markdown on disk.

Design pillars

- **One vault** — Scientia is FRIDAY's `VAULT_PATH`. Dev writes live in a dedicated `90-Meta/` zone.
- **Dev-owned zone** — FRIDAY *indexes* `90-Meta/` read-only and never writes or proposes into it, so playbook invariant **I1** (single writer) is preserved.
- **Two MCPs, no overlap** — FRIDAY's in-process SDK MCP (runtime) vs. the community `mcp-obsidian` server over the Local REST API plugin (dev-time Claude Code's hands on the vault).
- **Hybrid capture** — a deterministic *stub* written by a session-end hook (never misses, never hallucinates) plus a *narrative* written by the model on demand.
- **Machine-wide** — configured at user scope (`~/ .claude/`) so every project on this machine participates.

2. Combined repository & vault layout

The playbook's vault template gains one new top-level zone; the Next.js app gains one page. Additions are marked **NEW**.

```
Scientia/
  00-Inbox/ 10-Daily/ 15-Courses/ 30-Areas/ 99-Private/ # playbook template (writer.py-owned)
  90-Meta/
    sessions/2026/2026-07-12-friday-p0.md # auto stub + on-demand narrative
    projects/friday.md, projects/<other>.md # living per-project context page
    dashboard.md # map-of-content: recent sessions + projects
    _templates/{session.md, project.md}

friday/
  backend/journal.py # the app repo (built in P0)
  backend/main.py # NEW - read-only walk of 90-Meta/ (D1)
  web/journal/ # + /api/journal/* endpoints
  web/insights/ # NEW - Journal page (Section 5)
  ~/.claude/hooks/{session-start-context.ps1, session-end-journal.ps1} # + 'Development activity' strip
  ~/.claude/settings.json # + SessionStart / SessionEnd hooks
```

3. Dev-loop invariants (D1–D4)

These extend the playbook's **I1–I6** and win over convenience the same way. Violating any is a bug.

#	Invariant
D1	90-Meta/ is dev-owned . FRIDAY's <code>writer.py</code> and the suggest-then-approve loop never write or propose into it. This is what preserves I1 — FRIDAY still has exactly one writer for its own domain.
D2	FRIDAY indexes 90-Meta/ read-only for context, and all read surfaces (RAG, the Journal UI, the <code>/api/journal/*</code> endpoints) honor I3 privacy: 99-Private/ and <code>#no-ai</code> notes are excluded. A session note may be tagged <code>#no-ai</code> to keep it out.
D3	Session stubs are written by the hook via direct filesystem append and must work with Obsidian closed. Narrative writeups are written by the model via the Obsidian MCP, on demand only.
D4	Subscription auth only (I5 — never set <code>ANTHROPIC_API_KEY</code>). The Obsidian REST API key lives in the OS keyring / env, never committed (I4).

4. The full loop — data flow

- **Session start** — the `SessionStart` hook derives the project from the working directory and, if `90-Meta/projects/<project>.md` exists, injects it via `hookSpecificOutput.additionalContext` so Claude resumes grounded in prior decisions. Deterministic file read — no model, works offline.
- **During the session** — Claude pulls any *other* relevant vault notes on demand with the Obsidian MCP (`search`, `get_file_contents`).
- **Session end** — the `SessionEnd` hook appends an **objective stub** to today's session note: timestamp, project, cwd, `session_id`, and (when the cwd is a git repo) branch + `git diff --name-only`.
- **On demand** (“log this to Obsidian” / `/log-session`) — the model writes the **narrative** (what was done, decisions, next steps), appending to the same session note and updating the project note via the MCP.
- **Anytime** — you (or FRIDAY) read it back: in Obsidian, in FRIDAY's RAG answers, or in the **Journal UI** (Section 5).

The division of labor is the whole point: the **hook** captures what a script can know for free; the **model** captures meaning, only when asked; the **UI** makes both browsable without opening Obsidian.

5. Web UI surface — the Journal page & Insights

FRIDAY ships a Next.js 14 dashboard (App Router, Tailwind, SWR for data, native WebSocket for chat, recharts for insights, `obsidian://` deep links). The build journal is not just files — it is **presented in that same dashboard**, read-only, so you never have to open Obsidian to see it.

5.1 New nav item: Journal

A seventh page, `web/journal`, joins the existing six. Two-pane layout:

- **Left — Projects rail**: cards generated from `90-Meta/projects/*.md` (title, last-updated, session count, open-threads count).
- **Right — Session timeline**: reverse-chronological, grouped by day; each row shows project, git branch, files-changed count, and a badge for whether a narrative exists yet.
- **Detail pane**: clicking a session or project renders its markdown, with an `obsidian://open?vault=Scientia&file=...` chip to jump into Obsidian for editing.
- **Read-only by contract (D1)**: a subtle “dev-owned · view only” badge; edits happen in Obsidian or via `/log-session`. There are deliberately no journal write endpoints.

5.2 Insights integration

Rather than a standalone analytics page, the journal reuses the existing `recharts` setup on `web/insights` with a “Development activity” strip:

- Sessions-per-week bar chart; projects-touched count; files-changed trend (14 days).
- Sits alongside FRIDAY's own study/task insights — one dashboard, one visual language.

5.3 Backend (read-only, enforces D1/D2)

```
GET /api/journal/projects      -> [{slug, title, updated, sessions, open_threads}]
GET /api/journal/sessions?project=&range= -> [{date, project, branch, files, has_narrative, path}]
GET /api/journal/note?path=    -> {markdown, obsidian_uri}    # path confined to 90-Meta/
# No POST/PUT/DELETE for the journal. backend/journal.py walks 90-Meta/, skips
# 99-Private/ and #no-ai (D2), returns Pydantic models; CORS localhost:3000 only.
```

Conventions match the rest of the app: Pydantic response models, SWR on the client, path inputs are validated to stay inside 90-Meta/ (no traversal), and every read path applies the same privacy filter FRIDAY's RAG uses.

6. New roadmap phase

Slots after P0 (foundation) so the loop exists before P1 begins.

PHASE P0.5 — Build Journal & Obsidian Dev Loop

OBJECTIVE: stand up the Obsidian dev loop and its read-only UI so that building FRIDAY is journaled from day one. Independent of P1–P4; requires no FRIDAY agent/RAG code.

BUILD:

- Vault: scaffold 90-Meta/{sessions/2026/, projects/, dashboard.md, _templates/} in Scientia; seed projects/friday.md.
- MCP: enable the Local REST API plugin; register obsidian at user scope — `claude mcp add obsidian -s user -e OBSIDIAN_API_KEY=... -e OBSIDIAN_PORT=27124 -- uvx mcp-obsidian`.
- Hooks (~/.claude/hooks/*.ps1): `session-start-context.ps1` (inject matching project note) and `session-end-journal.ps1` (append objective stub); wire both into ~/.claude/settings.json.
- Command: `/log-session` — model composes the narrative and updates session + project notes via the MCP, honoring D2.
- Backend: `backend/journal.py` + `/api/journal/*` read-only endpoints.
- Web: `web/journal` page + nav entry (now 7 pages); 'Development activity' strip on `web/insights`.

EXIT CRITERIA (run and paste output):

- `claude mcp list` -> obsidian connected; an MCP search over Scientia returns `Welcome.md` (Obsidian running).
- Run `session-end-journal.ps1` with a sample payload -> a stub note appears under 90-Meta/sessions/2026/ with correct git fields.
- Seed 90-Meta/projects/<repo>.md, start a session in that repo -> its contents are injected (SessionStart additionalContext); end it -> stub appended for the right project.
- `npm run build` compiles; nav renders 7 pages incl. Journal; Journal lists the seeded project + session; clicking opens the rendered note with a working obsidian:// link.
- `pytest`: a `#no-ai` note never appears in any `/api/journal/*` response (D2); journal path inputs cannot escape 90-Meta/.
- `/log-session` -> narrative lands in today's note and the project note updates, with no writes outside 90-Meta/ (grep proof).

7. Technology cheat-sheet additions

Component	Choice	Implementation notes
Obsidian bridge	Local REST API plugin	Community plugin in Scientia; generate API key; default port 27124 (HTTPS). Must be running for MCP reads/searches.
Dev-time MCP	mcp-obsidian (uvx)	<code>uvx mcp-obsidian</code> ; <code>env OBSIDIAN_API_KEY / HOST(127.0.0.1) / PORT(27124)</code> ; 7 tools: list, get, search, patch-by-heading, append, delete.
Session capture	Claude Code hooks	SessionStart (matcher startup resume; additionalContext) + SessionEnd; PowerShell scripts on Windows; user-scope <code>~/.claude/settings.json</code> .
Runtime	uv / uvx	Prereq for the MCP server. <code>winget install astral-sh.uv</code> if missing.
Journal UI	Next.js 14 + recharts	Reuses the app's App Router + Tailwind + SWR + recharts; read-only; <code>obsidian://</code> deep links (unchanged from playbook stack).

8. Setup & expected human touchpoints

Most of P0.5 is autonomous. The steps only you can do:

- Install + enable the **Local REST API** plugin in Obsidian (Scientia) and generate the API key; keep Obsidian running for MCP reads.
- Ensure **uv** is installed (`winget install astral-sh.uv`) so `uvx` is available.
- Provide the API key for `claude mcp add` (or it is registered with a placeholder for you to fill).

Everything else — scaffolding, hooks, endpoints, the Journal page — is Claude Code's job. If it hits a question this spec or the playbook already answers, the standing reply is: "Decide per the spec and log it in BUILD_LOG.md."

This addendum defines HOW the dev loop attaches to FRIDAY; the Autonomous Build Playbook v1.0 and the v0.2 spec remain authoritative for the rest. Where they disagree: the spec defines WHAT, the playbook defines HOW, the Master Prompt invariants (I1–I6) and these dev-loop invariants (D1–D4) win over both.